ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ «САМАРСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИМЕНИ АКАДЕМИКА С.П. КОРОЛЕВА

Лекции по курсу «Нейронные сети и машинное обучение»

Для студентов направления 090401

Очное обучение 2026-2027 учебный год

Лектор: Солдатова Ольга Петровна, к.т.н., доцент

Модели машинного обучения

Лекция 2.

Математические модели машинного обучения

Обзор методов классификации в машинном обучении с помощью Scikit-Learn.

Реализация классификатора.

Метод «k-ближайших соседей».

Случайный лес.

Теорема Байеса.

Наивный классификатор Байеса.

Литература по курсу

1. Куприянов А.В. «Искусственный интеллект и машинное обучение» https://do.ssau.ru/moodle/course/view.php?id=1459
2. Осовский С. Нейронные сети для обработки информации / Пер. с пол. И.Д. Рудинского. – М.: Финансы и статистика, 2002. – 344 с.: ил.
3. Горбаченко, В. И. Интеллектуальные системы: нечеткие системы и сети : учебное пособие для вузов / В. И. Горбаченко, Б. С. Ахметов, О. Ю. Кузнецова. — 2-е изд., испр. и доп. — Москва : Издательство Юрайт, 2018. — 105 с. — (Университеты России). — ISBN 978-5-534-08359-0. — Текст : электронный // ЭБС Юрайт [сайт]. — URL: https://urait.ru/bcode/424887 – Режим доступа: https://urait.ru/bcode/424887
4. 4. Гафаров Ф.М. Искусственные нейронные сети и приложения: учеб. пособие / Ф.М. Гафаров, А.Ф. Галимянов. – Казань: Изд-во Казан. ун-та, 2018. – 121 с. – Текст : электронный. – Режим доступа: https://repository.kpfu.ru/?p_id=187099
5. Хайкин С. Нейронные сети: Полный курс: Пер. с англ. - 2-е изд. – М.: Вильямс, 2006. – 1104 с.: ил.
6. Борисов В.В., Круглов В.В., Федулов А.С. Нечёткие модели и сети. – М.: Горячая линия– Телеком, 2007. -284 с.: ил.
7. Рутковская Д., Пилиньский М., Рутковский Л. Нейронные сети, генетические алгоритмы и нечёткие системы: Пер. с польск. И.Д.Рудинского, - М.: Горячая линия – Телеком, 2007. – 452 с. ил.
8. Николенко С., Кадурин А., Архангельская Е. Глубокое обучение. — СПб.: Питер, 2018. — 480 с.: ил. — (Серия ≪Библиотека программиста≫).
9. Гудфеллоу Я., Бенджио И., Курвилль А. Глубокое обучение / пер. с анг. А. А. Слинкина. – 2-е изд., испр. – М.: ДМК Пресс, 2018. – 652 с.: цв. ил.

Математические модели машинного обучения

Придумано большое количество моделей машинного обучения, простых и сложных, точных и не очень, легко вычисляемых и требующих огромных вычислительных мощностей.

Поиск лучшей модели — это сложный эксперимент, надо перебрать разные модели, разные варианты моделей, а все это требует времени и средств.

Нельзя заранее сказать какая модель будет лучше или хуже, бывает так, что простая модель окажется лучше сложной или наоборот. В первую очередь это зависит от данных, на которых модель обучается.

Есть множество простых задач, которые уже многократно успешно решались, но можно изменить условия и модель перестанет работать. Например, обучили модель распознавать автомобили, но изменилось освещение, пошел дождь, образовался туман, и модель не работает.

Модели машинного обучения

Основные математические модели машинного обучения:

- деревья решений и случайный лес;
- наивная байесовская классификация;
- линейная регрессия и метод наименьших квадратов;
- логистическая регрессия;
- метод опорных векторов;
- линейный дискриминант Фишера;
- ансамблевые методы: бустинг, беггинг, конкретно xgboost, catboost;
- метод «к-средних» Kmeans;
- метод «к ближайших соседей» KNN;
- метод главных компонент PCA;
- нейронные сети.

Эвристические алгоритмы повышения точности оценок моделей машинного обучения

***Бустинг* (boosting)**

- ✓ Базовые алгоритмы строятся последовательно, для каждого базового алгоритма, начиная со второго, веса обучающих объектов пересчитываются так, чтобы он точнее настраивался на тех объектах, на которых чаще ошибались все предыдущие алгоритмы. Веса алгоритмов также вычисляются исходя из числа допущенных ими ошибок.
- ✓ Бустинг представляет собой жадный алгоритм построения композиции алгоритмов.
- ✓ Наиболее известен алгоритм AdBoost предложенный в 1995 г. Шапиро (Schapire R. The boosting approach to machine learning: An overview // MSRI Workshop on Nonlinear Estimation and Classification, Berkeley, CA. - 2001.)

***Баггинг* (bagging — «bootstrap aggregation»)**

- ➢ В отличие от *бустинга* все элементарные классификаторы обучаются и работают параллельно (независимо друг от друга). Идея заключается в том, что классификаторы не исправляют ошибки друг друга, а компенсируют их при голосовании.
- ➢ Предложен Л. Брейманом (Breiman L. Arcing classifiers // The Annals of Statistics. – 1998. – Vol. 26, no. 3. – Pp. 801−849.), производится взвешенное голосование базовых алгоритмов, обученных на различных подвыборках данных, либо на различных частях признакового описания объектов.
- ➢ Базовые классификаторы должны быть независимыми, это могут быть классификаторы основанные на разных группах методов или же обученные на независимых наборах данных. Во втором случае можно использовать один и тот же метод.
- ➢ Из множества объектов обучающей выборки формируются *бутстреп- выборки* – случайные подмножества объектов, как правило с повторами.

Обзор методов классификации в машинном обучении с помощью Scikit-Learn

Scikit-Learn — это Python-библиотека, впервые разработанная David Cournapeau в 2007 году. В этой библиотеке находится большое количество алгоритмов для задач, связанных с классификацией и машинным обучением в целом.

Scikit-Learn базируется на библиотеке SciPy, которую нужно установить перед началом работы.

В системах машинного обучения или же системах нейросетей существуют входы и выходы. То, что подаётся на входы, принято называть признаками (англ. features).

Признаки по существу являются тем же, что и переменные в научном эксперименте — они характеризуют какой-либо наблюдаемый феномен и их можно как-то количественно измерить.

Когда признаки подаются на входы системы машинного обучения, эта система пытается найти совпадения, заметить закономерность между признаками. На выходе генерируется результат этой работы.

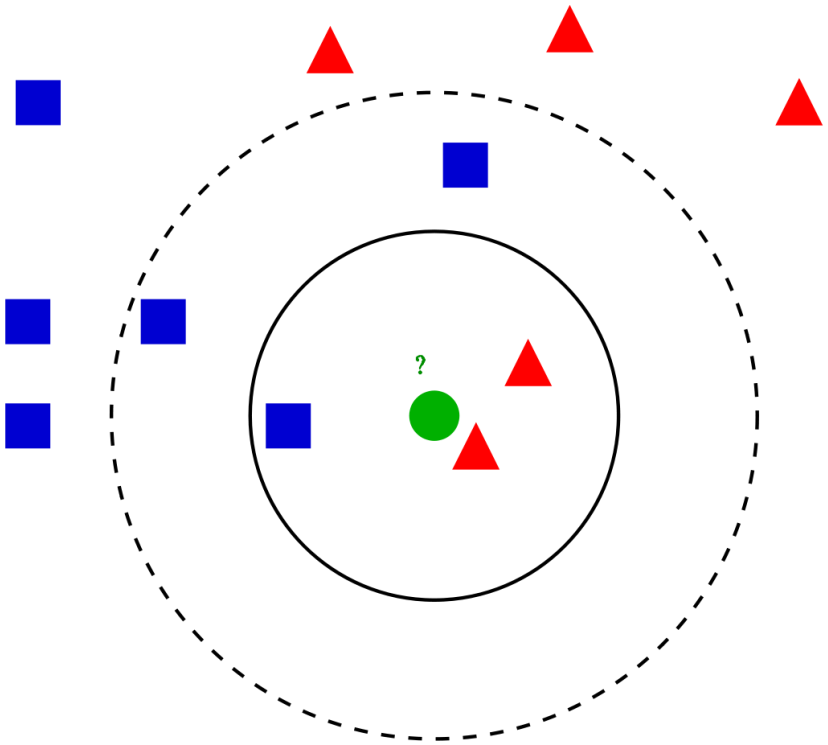
Обзор методов классификации в машинном обучении с помощью Scikit-Learn

Этот результат принято называть меткой (англ. label), поскольку у выходов есть некая пометка, выданная им системой, т. е. предположение (прогноз) о том, в какую категорию попадает выход после классификации.

Scikit-Learn даёт доступ ко множеству различных алгоритмов классификации. Вот основные из них:

- ✓ Метод k-ближайших соседей (K-Nearest Neighbors);
- ✓ Метод опорных векторов (Support Vector Machines);
- ✓ Классификатор дерева решений (Decision Tree Classifier) / Случайный лес (Random Forests);
- ✓ Наивный байесовский метод (Naive Bayes);
- ✓ Линейный дискриминантный анализ (Linear Discriminant Analysis);
- ✓ Логистическая регрессия (Logistic Regression);

Метод k-ближайших соседей (K-Nearest Neighbors)



Этот метод работает с помощью поиска кратчайшей дистанции между тестируемым объектом и ближайшими к нему классифицированным объектами из обучающего набора. Классифицируемый объект будет относится к тому классу, к которому принадлежит ближайший объект набора.

Другие классификаторы

Классификатор дерева решений (Decision Tree Classifier). Этот классификатор разбивает данные на всё меньшие и меньшие подмножества на основе разных критериев, т. е. у каждого подмножества своя сортирующая категория. С каждым разделением количество объектов определённого критерия уменьшается.

Классификация подойдёт к концу, когда сеть дойдёт до подмножества только с одним объектом. Если объединить несколько подобных деревьев решений, то получится так называемый Случайный Лес (англ. Random Forest).

Наивный байесовский классификатор (Naive Bayes).

Такой классификатор вычисляет вероятность принадлежности объекта к какому-то классу. Эта вероятность вычисляется из шанса, что какое-то событие произойдёт, с опорой на уже на произошедшие события.

Каждый параметр классифицируемого объекта считается независимым от других параметров.

Другие классификаторы

Линейный дискриминантный анализ (Linear Discriminant Analysis).

Этот метод работает путём уменьшения размерности набора данных, проецируя все точки данных на линию. Потом он комбинирует эти точки в классы, базируясь на их расстоянии от центральной точки.

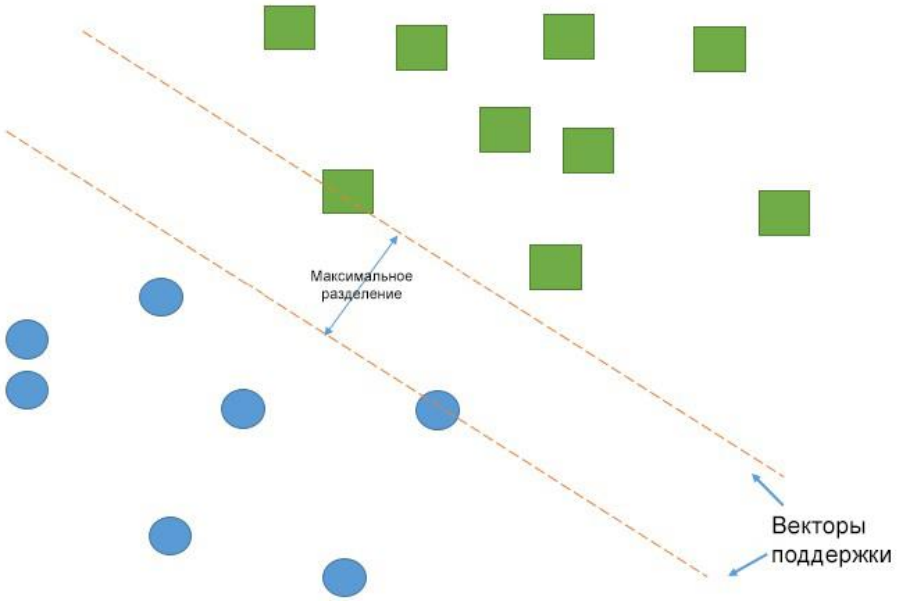
Этот метод, как можно уже догадаться, относится к линейным алгоритмам классификации, т. е. он хорошо подходит для данных с линейной зависимостью.

Логистическая регрессия (Logistic Regression).

Логистическая регрессия выводит прогнозы о точках в бинарном масштабе — нулевом или единичном. Если значение чего-либо равно либо больше 0,5, то объект классифицируется в большую сторону (к единице). Если значение меньше 0,5 — в меньшую (к нулю).

У каждого признака есть своя метка, равная только 0 или только 1. Логистическая регрессия является линейным классификатором и поэтому используется, когда в данных прослеживается какая-то линейная зависимость.

Метод опорных векторов (Support Vector Machines)



Работа метода опорных векторов заключается в рисовании линии между разными кластерами точек, которые нужно сгруппировать в классы. С одной стороны линии будут точки, принадлежащие одному классу, с другой стороны — к другому классу.

Классификатор будет пытаться увеличить расстояние между рисуемыми линиями и точками на разных сторонах, чтобы увеличить свою «уверенность» определения класса. Когда все точки построены, сторона, на которую они падают — это класс, которому эти точки принадлежат.

Реализация классификатора

Первый шаг в реализации классификатора — его импорт в Python. Вот как это выглядит:

```
from sklearn.linear_model import LogisticRegression
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.tree import DecisionTreeClassifier
from sklearn.svm import SVC

# Импорт всех нужных библиотек
import pandas as pd
import numpy as np
from sklearn.metrics import classification_report
from sklearn.metrics import confusion_matrix
from sklearn.metrics import accuracy_score
```

Реализация классификатора

Исходные данные встроены в библиотеку Scikit-learn, скачиваются и разделяются на обучающую и тестовую выборки при помощи следующих команд языка Python:

```
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
data = load_wine()
X = data.data
Y = data.targe
```

Посмотреть данные можно следующим образом:

```
print('Всего образцов', X.shape[0])
print('Кол-во признаков', X.shape[1])
print('Признаки')
print(X)
print('Метки')
print(Y)
```

После этого надо построить диаграмму рассеивания.

Реализация классификатора

Процесс классификации содержит в себе следующие этапы: подготовка данных, создание обучающих наборов, создание классификатора, обучение классификатора, составление прогнозов, оценка производительности классификатора и настройка параметров.

Отсутствие значений в данных либо любые другие отклонения — все их нужно обработать, иначе они могут негативно влиять на производительность классификатора. Этот этап называется предварительной обработкой данных (англ. data preprocessing).

Затем следует выделить отфильтровать данные в соответствии с заданием и разделить все данные на обучающие и тестовые, например. так:

```
X_train, X_test, Y_train, Y_test = train_test_split (X_filtered, Y_filtered, test_size=0.2, random_state=42)
```

После этого нужно создать экземпляр классификатора. Сделать это можно создав переменную и вызвав функцию, связанную с классификатором.

```
logreg_clf = LogisticRegression()
```

Реализация классификатора

Во-первых, нужно подготовить набор данных для классификатора — преобразовать данные в корректную для классификации форму и обработать любые аномалии в этих данных.

Теперь классификатор нужно обучить. Перед этим нужно «подогнать» его под тренировочные данные.

Обучающие признаки и метки помещаются в классификатор через функцию fit.

```
logreg_clf.fit(X_trane,Y_traine).
```

Эти этапы (создание экземпляра, обучение и классификация) являются основными при работе с классификаторами в Scikit-Learn.

После этих шагов модель можно тестировать. Это можно сделать через функцию классификатора predict, передав ей параметр (признак) для прогнозирования:

```
logreg_clf.predict(test_features).
```

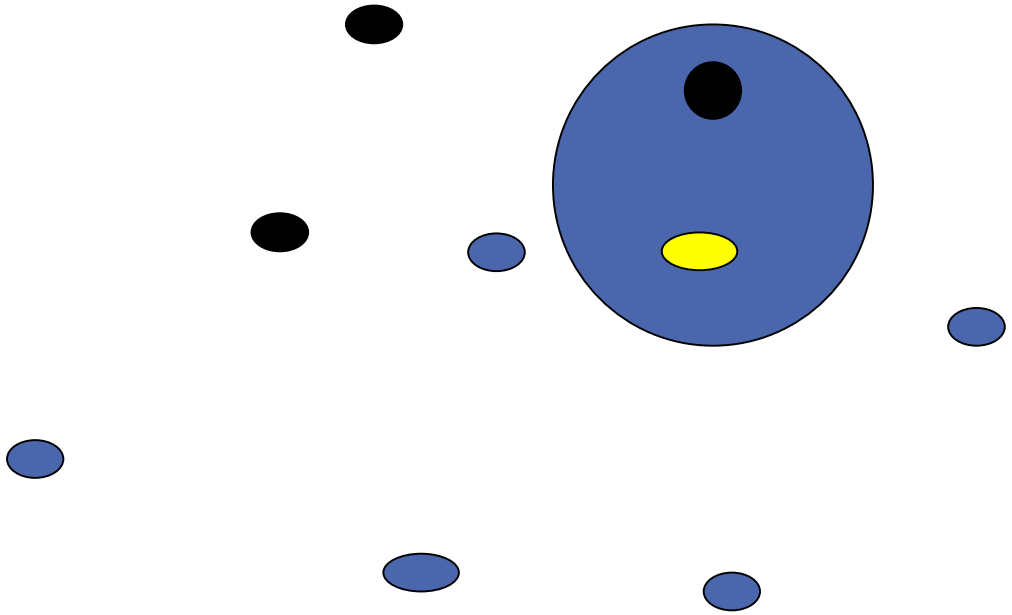
Сравнивая показания классификатора с фактически известными тестовыми данными, можно делать вывод о точности классификатора.

Метод «k-ближайших соседей»

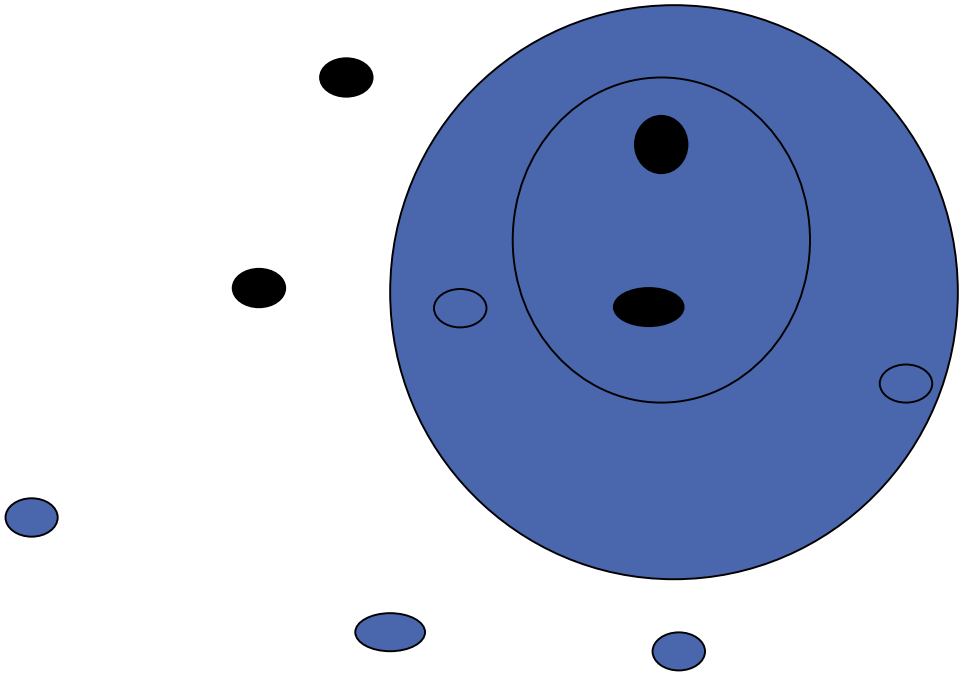
K-nearest neighbor – kNN

- Метод решения задачи классификации, который относит объекты к классу, которому принадлежит большинство из *k* его ближайших соседей в многомерном пространстве признаков.
- Число *k* – это количество соседних объектов в пространстве признаков, которое сравнивается с классифицируемым объектом.
- Использование только одного ближайшего соседа (1NN) ведёт к ошибкам из-за:
  - нетипичных примеров.
  - ошибок в ручной привязке единственного обучающего примера.
- Более устойчивой альтернативой является *k* наиболее похожих примеров и определение большинства
- Величина *k* типично нечётная: 3, 5

1-ближайший сосед



3-ближайших соседа



Нормализация и вычисление расстояния

Методы нормализации признаков обьектов:

- Минимаксная нормализация: $x^* = \frac{x - x_{min}}{x_{max} - x_{min}}$
- Нормализация с помощью стандартного отклонения: $x^* = \frac{x - x_{mean}}{\sigma_x}$
- где σ_x - стандартное отклонение.

Метрики расстояния:

- Евклидово расстояние: $\rho(x_1, x_2) = \sqrt{\sum_{i=1}^{n} (x_{1,i} - x_{2,i})^2}$
- Расстояние Чебышёва: $\rho(x_1, x_2) = \max_{i=1,\ldots,n} |x_{1,i} - x_{2,i}|$
- Манхэттенское расстояние: $\rho(x_1, x_2) = \sum_{i=1}^{n} |x_{1,i} - x_{2,i}|$

Достоинства метода kNN

Достоинства:

- ✓ Программная реализация относительно проста.
- ✓ Возможность модификации алгоритма.
- ✓ Алгоритм устойчив к аномальным выбросам.
- ✓ Возможность интерпретации результатов работы алгоритма.

Недостатки:

- ➢ Набор данных, используемый для алгоритма, должен быть репрезентативным.
- ➢ Необходимость хранить обучающую выборку целиком.
- ➢ В простейших случаях метрические алгоритмы имеют крайне бедный набор параметров, что исключает возможность настройки алгоритма по данным.
- ➢ Затраты в производительности велики, поскольку необходимо вычислить расстояния между каждым экземпляром и всеми пробными экземплярами.

Пример: Ирисы Фишера

150 цветков трех классов:

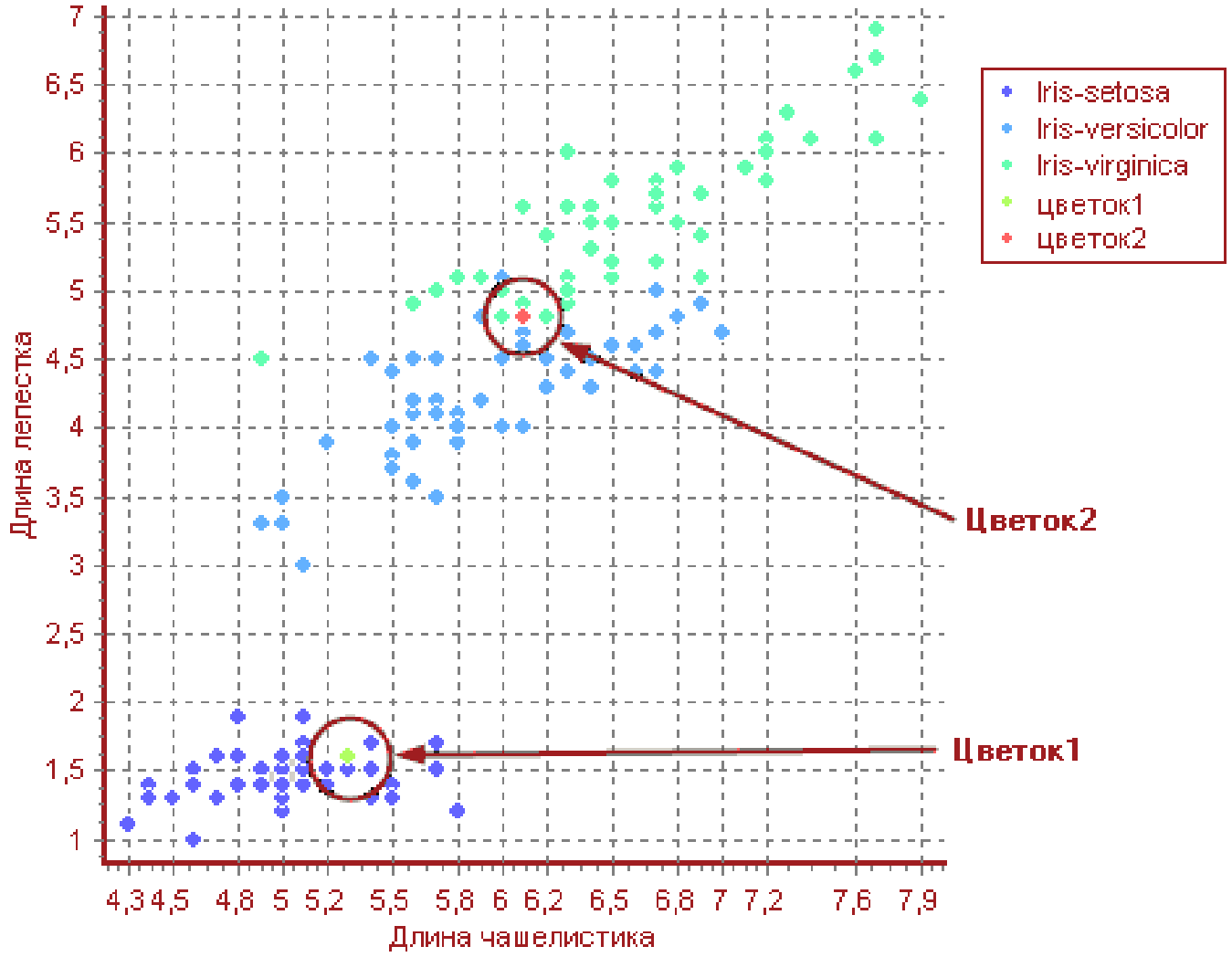




Два параметра: длина чашелистика и длина лепестка.

Два новых цветка со следующими значениями длины чашелистика и лепестка: 5,3 и 1,6 (***цветок 1***), 6,1 и 4,8 (***цветок 2***).

Ирисы Фишера: диаграмма размещения классов



Ирисы Фишера: простое голосование

Цветок 1. Зададим k=3.

Ближайшие соседи: *A* (5,3; 1,5), *B*(5,2; 1,5) и *C*(5,2; 1,5).

$$d(\text{цветок } 1, A) = \sqrt{(5{,}3 - 5{,}3)^2 + (1{,}6 - 1{,}5)^2} = 0{,}1$$

$$d(\text{цветок } 1, B) = \sqrt{(5{,}3 - 5{,}2)^2 + (1{,}6 - 1{,}5)^2} = 0{,}14$$

$$d(\text{цветок } 1, \text{C}) = \sqrt{(5{,}3 - 5{,}2)^2 + (1{,}6 - 1{,}5)^2} = 0{,}14$$

Объект	Чашелистик	Лепесток	Расстояние	Класс
Цветок 1	5,3	1,6	-	-
A	5,3	1,5	0,1	Iris Setosa
B	5,2	1,5	0,14	Iris Setosa
C	5,2	1,5	0,14	Iris Setosa

Класс *цветка 1*: *Iris Setosa*

Ирисы Фишера: простое голосование

Цветок 2. Зададим k=3 и предположим, что длина лепестка вдвое важнее длины чашелистика.

Ближайшие соседи: $A(6{,}1;\ 4{,}7)$, $B(6;\ 4{,}8)$, $C(6{,}2\ 4{,}8)$

$$d(\text{цветок } 2, A) = \sqrt{(6{,}1 - 6{,}1)^2 + 2(4{,}8 - 4{,}7)^2} = 0{,}14$$

$$d(\text{цветок } 2, B) = \sqrt{(6{,}1 - 6)^2 + 2(4{,}8 - 4{,}8)^2} = 0{,}1$$

$$d(\text{цветок } 2, C) = \sqrt{(6{,}1 - 6{,}2)^2 + 2(4{,}8 - 4{,}8)^2} = 0{,}1$$

Объект	Чашелистик	Лепесток	Расстояние	Класс
Цветок 2	6,1	4,8	-	-
A	6,1	4,7	0,14	Iris Versicolour
B	6	4,8	0,1	Iris Virginica
C	6,2	4,8	0,1	Iris Virginica

Класс *цветка 2*: *Iris Virginica*

Деревья решений

Деревья решений – *способ представления правил в иерархической, последовательной структуре*.

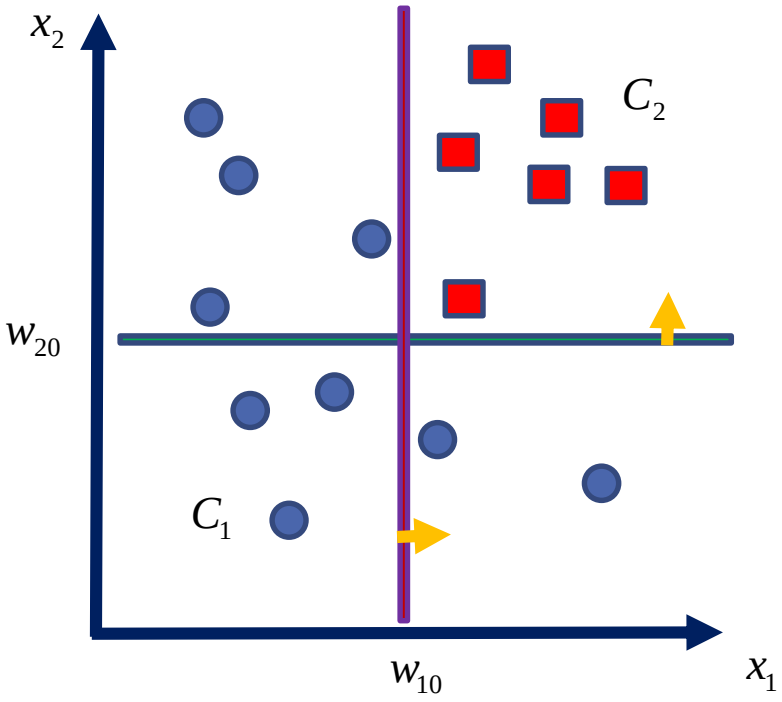
Основополагающая работа – книга Ханта (Hunt, E.B.), Мэрина (Marin J.) и Стоуна (Stone, P.J) "Experiments in Induction", увидевшая свет в 1966г.

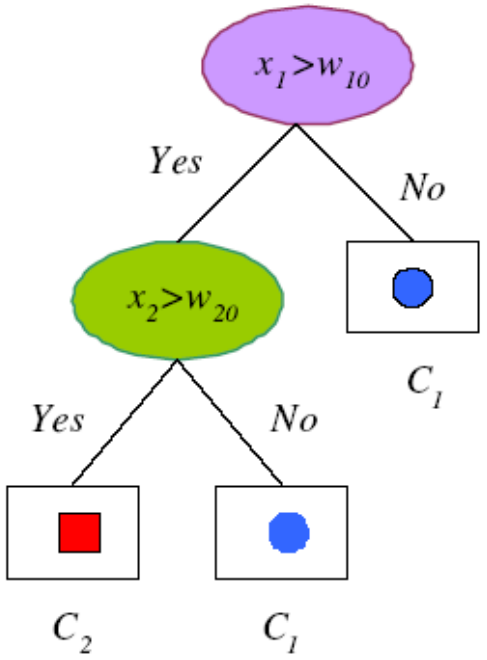
Метод также называют деревьями решающих правил, деревьями классификации и регрессии.

Определение:

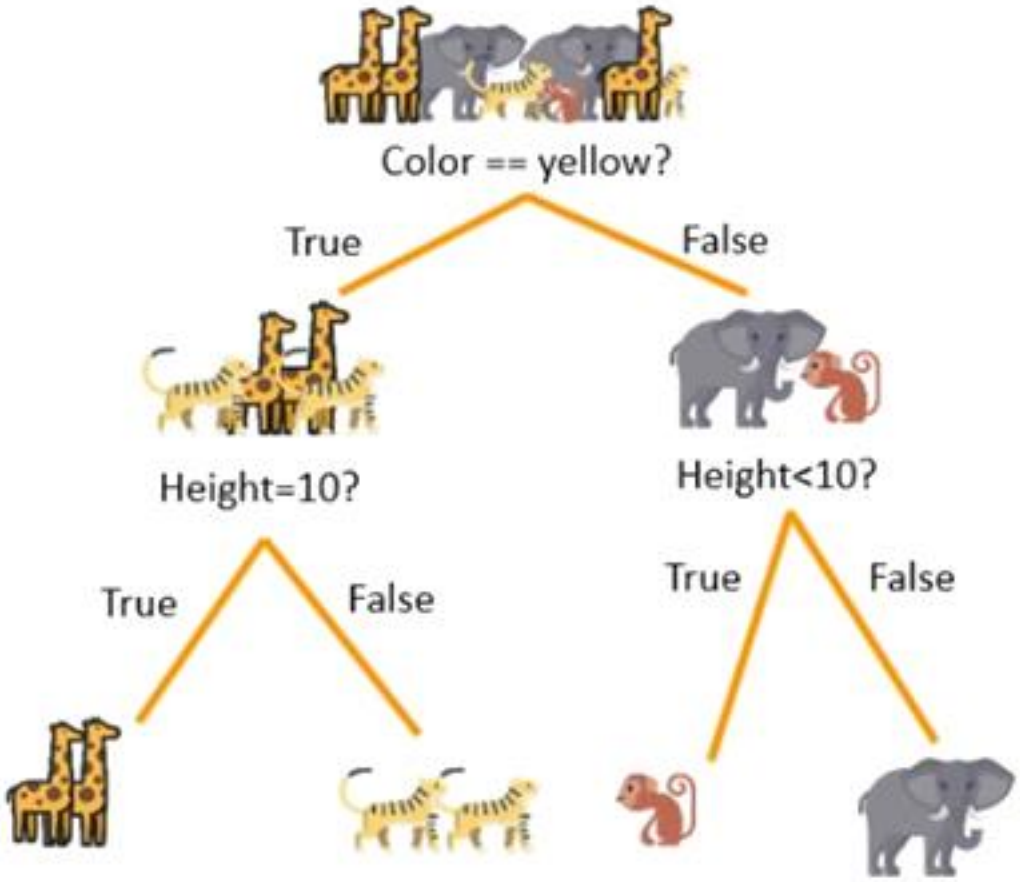
Дерево решений – представленный в виде связного ациклического графа план, при помощи которого оценивается значение целевого атрибута объекта по набору значений независимых атрибутов.

Пример





Деревья решений



Деревья решений

Дерево принятия решений (также называют деревом классификации или регрессионным деревом) — средство поддержки принятия решений, использующееся в машинном обучении, анализе данных и статистике. Структура дерева представляет собой «листья» и «ветки». На рёбрах («ветках») дерева решения записаны признаки, от которых зависит целевая функция, в «листьях» записаны значения целевой функции, а в остальных узлах — признаки, по которым различаются случаи. Чтобы классифицировать новый случай, надо спуститься по дереву до листа и выдать соответствующее значение.

Подобные деревья решений широко используются в интеллектуальном анализе данных. Цель состоит в том, чтобы создать модель, которая предсказывает значение целевой переменной на основе нескольких переменных на входе.

Каждый лист представляет собой значение целевой переменной, изменённой в ходе движения от корня по рёбрам дерева до листа. Каждый внутренний узел сопоставляется с одной из входных переменных.

Деревья решений

Дерево может быть также «изучено» разделением исходных наборов переменных на подмножества, основанные на проверке значений признаков. Это действие повторяется на каждом из полученных подмножеств. Рекурсия завершается тогда, когда подмножество в узле имеет те же значения целевой переменной, таким образом, оно не добавляет ценности для предсказаний. Процесс, идущий «сверху вниз», индукция деревьев решений (TDIDT)[1], является примером поглощающего «жадного» алгоритма, и на сегодняшний день является наиболее распространённой стратегией деревьев решений для данных, но это не единственная возможная стратегия.

В интеллектуальном анализе данных деревья решений могут быть использованы в качестве математических и вычислительных методов, чтобы помочь описать, классифицировать и обобщить набор данных, которые могут быть записаны следующим образом:

$$(x, Y) = (x_1, x_2, ..., x_k, Y)$$

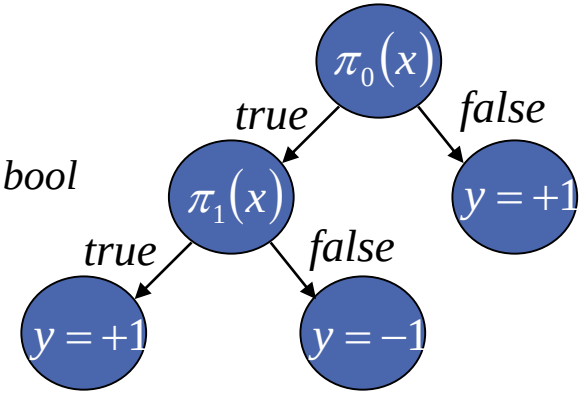
Зависимая переменная Y является целевой переменной, которую необходимо проанализировать, классифицировать и обобщить. Вектор x состоит из входных переменных $x_1, x_2, ..., x_k$ которые используются для выполнения этой задачи.

Деревья решений

- ✓ Двоичное дерево.
- ✓ Узлы – помечены некоторым предикатом $\pi : X \to bool$
- ✓ Обычно, используются пороги по проекциям на оси координат (на элементы вектора признаков)

$$\pi\left(x = (x^0, \ldots x^m)\right) = \begin{cases} true, & x^j < \Theta \\ false, & x^j \geq \Theta \end{cases}$$

- ✓ Связи – помечены $\{true \quad false\}$.
- ✓ Листья – помечены ответами из *Y*.



- ➢ **Внутренний узел** представляет разбиение множества возможных значений той или иной независимой переменной
- ➢ Атрибуты, соответствующие внутренним узлам дерева – **атрибуты расщепления** (прогнозирующие атрибуты).
- ➢ Каждая ветвь от внутреннего узла отмечается **предикатом расщепления**.
- ➢ Информация об атрибутах и предикатах расщепления в узле – **критерий расщепления**.

Преимущества:

- ❖ **Интуитивность деревьев решений**.
- ❖ Возможность извлекать правила из базы данных на **естественном языке.**
- ❖ **Не требует от пользователя выбора входных атрибутов.**
- ❖ **Точность** моделей.
- ❖ Разработан ряд **масштабируемых алгоритмов.**
- ❖ **Быстрый процесс обучения** .
- ❖ Обработка **пропущенных значений.**
- ❖ Работа и с числовыми, и с **категориальными** типами данных .

Деревья решений

Модель алгоритма обучения

Дана обучающая выборка

1. Выбираем целевой атрибут.
2. Выбираем критерий расщепления.
3. Разделяем обучающую выборку.
4. Исключаем атрибут расщепления из выборки.
5. Для всех полученных подвыборок переходим на шаг 2.

Модель классификатора

- ✓ Выходом классификатора $a(x)$ является значение y листа, полученного при обходе:
  - ✓ Начинаем от корня.
  - ✓ Переходим в тот узел, в который ведёт связь помеченная значением предиката в текущем узле.
  - ✓ Заканчиваем, попав в лист.

Лист определяет собой область пространства *X*

Если зависимая переменная принимает дискретные значения – задача классификации.

Если непрерывные – задачу регрессии (численного прогнозирования).

Построение дерева решений

- Процесс заключается в последовательном, рекурсивном разбиении обучающего множества на подмножества с применением решающих правил в узлах.
  - Выбираем очередной атрибут ***Q***, помещаем его в корень.
  - Для всех его значений ***i***:
    - Оставляем из тестовых примеров только те, у которых значение атрибута ***Q*** равно ***i***
    - Рекурсивно строим дерево в этом потомке.
- Процесс разбиения продолжается до тех пор, пока все узлы в конце всех ветвей не будут объявлены листьями.
  - Объявление узла листом может произойти естественным образом (когда он будет содержать единственный объект, или объекты только одного класса), или по достижении некоторого условия остановки, задаваемого пользователем (например, минимально допустимое число примеров в узле или максимальная глубина дерева).

Особенности построения деревьев решений

- Алгоритмы построения деревьев решений относят к категории так называемых жадных алгоритмов.
  - Жадными называются алгоритмы, которые допускают, что локально-оптимальные решения на каждом шаге (разбиения в узлах), приводят к оптимальному итоговому решению.
  - В случае деревьев решений это означает, что если один раз был выбран атрибут, и по нему было произведено разбиение на подмножества, то алгоритм не может вернуться назад и выбрать другой атрибут, который дал бы лучшее итоговое разбиение. Поэтому на этапе построения нельзя сказать обеспечит ли выбранный атрибут, в конечном итоге, оптимальное разбиение.

Основные этапы:

- Выбор атрибута, по которому будет производиться разбиение в данном узле (атрибута разбиения).
- Выбор критерия остановки обучения.
- Выбор метода отсечения ветвей (упрощения).
- Оценка точности построенного дерева.

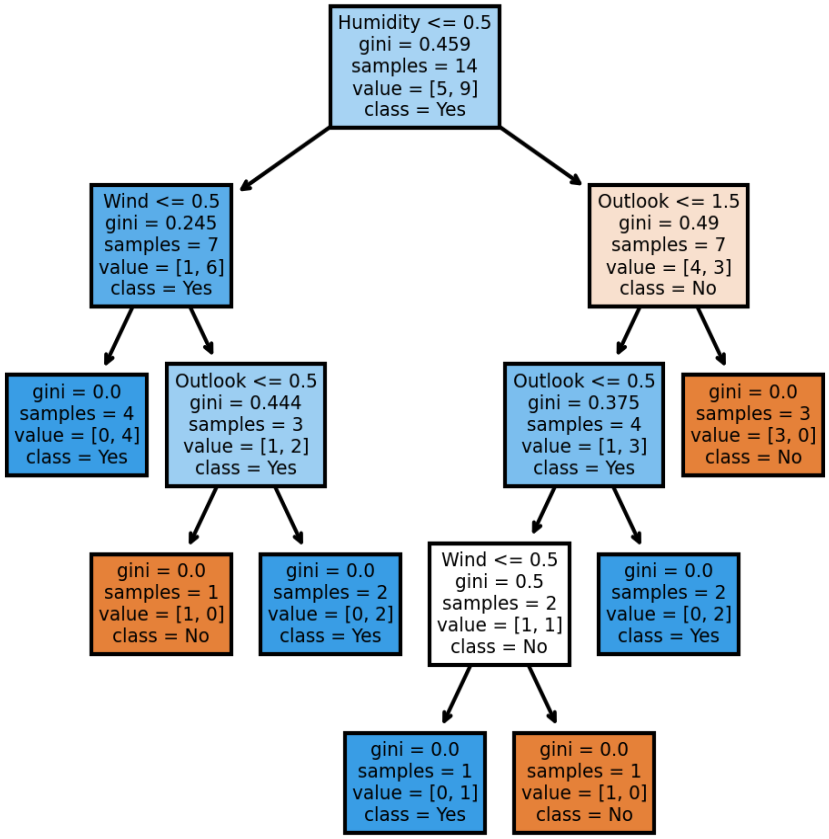
Алгоритмы обучения

- **ID3 (Iterative Dichotomizer 3)** — алгоритм позволяет работать только с дискретной целевой переменной, поэтому деревья решений, построенные с помощью данного алгоритма, являются классифицирующими. Число потомков в узле дерева не ограничено. Не может работать с пропущенными данными.
- **C4.5** — усовершенствованная версия алгоритма ID3, в которую добавлена возможность работы с пропущенными значениями атрибутов *по версии издания Springer Science в 2008 году алгоритм занял 1-е место в топ-10 наиболее популярных алгоритмов Data Mining*.
- **CART (Classification and Regression Tree)** — алгоритм обучения деревьев решений, позволяющий использовать как дискретную, так и непрерывную целевую переменную, то есть решать как задачи классификации, так и регрессии. Алгоритм строит деревья, которые в каждом узле имеют только два потомка.

Деревья решений. Пример.

PlayTennis: training examples

Day	Outlook	Temperature	Humidity	Wind	PlayTennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No



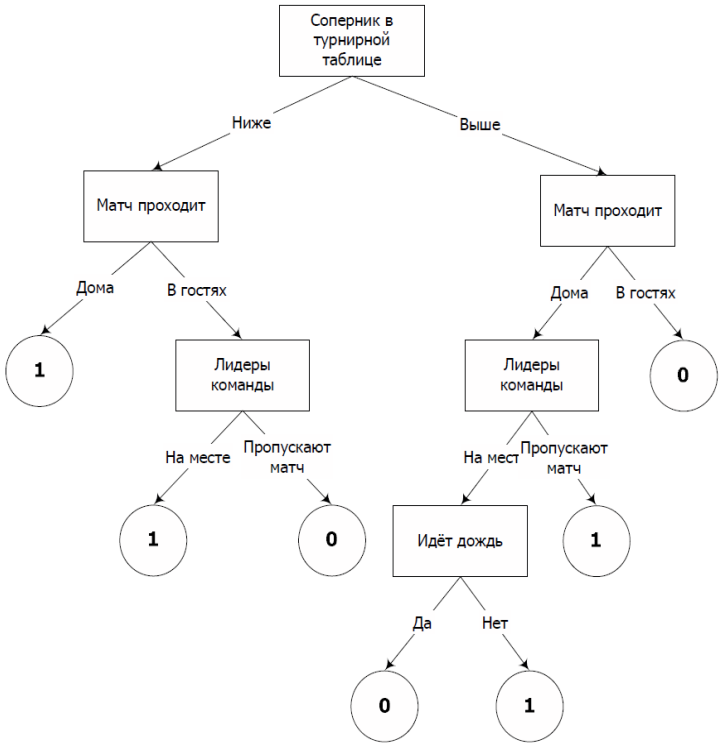
x=(Sunny, Cool, High, Strong)

Критерии остановки алгоритма

- Если «рост» дерева не ограничить, то в результате будет построено сложное дерево с большим числом узлов и листьев.
  - Проблема переобучения — точное распознавание примеров, участвующих в обучении и полная несостоятельность на новых данных. Переобученные деревья имеют очень сложную структуру, и поэтому их сложно интерпретировать.
- Ранняя остановка — алгоритм будет остановлен, как только будет достигнуто заданное значение некоторого критерия, например процентной доли правильно распознанных примеров.
  - Единственным преимуществом подхода является снижение времени обучения. Главным недостатком является то, что ранняя остановка всегда делается в ущерб точности дерева, поэтому многие авторы рекомендуют отдавать предпочтение отсечению ветвей.
- Ограничение глубины дерева — задание максимального числа разбиений в ветвях, по достижении которого обучение останавливается.
  - Данный метод также ведёт к снижению точности дерева.
- Задание минимально допустимого число примеров в узле — запретить алгоритму создавать узлы с числом примеров меньше заданного (например, 5).
  - Это позволит избежать создания тривиальных разбиений и, соответственно, малозначимых правил.

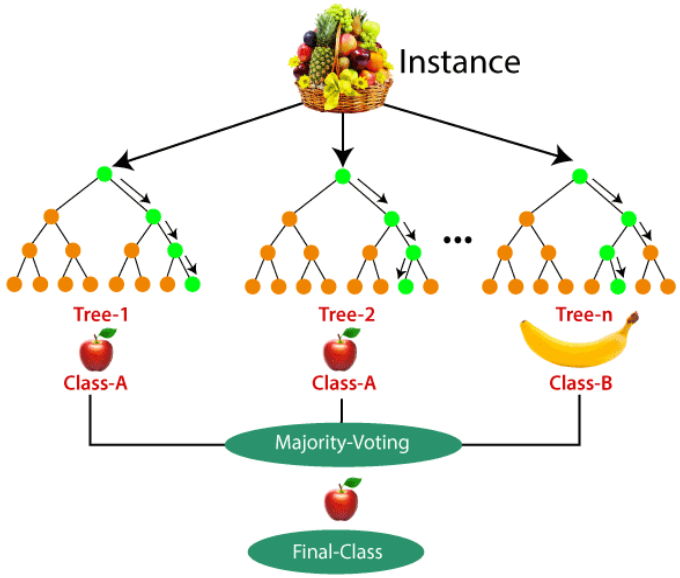
Пример построения дерева

- Как сыграет команда?



Соперник	Играем	Лидеры	Дождь	Победа
Выше	Дома	На месте	Да	Нет
Выше	Дома	На месте	Нет	Да
Выше	Дома	Пропускают	Нет	Да
Ниже	Дома	Пропускают	Нет	Да
Ниже	В гостях	Пропускают	Нет	Нет
Ниже	Дома	Пропускают	Да	Да
Выше	В гостях	На месте	Да	Нет
Ниже	В гостях	На месте	Нет	*???*

Случайный лес



- Метод случайного леса (англ. **random forest**) — алгоритм машинного обучения (предложен в 2001 г.), сочетает в себе две основные идеи: метод бэггинга (решающие деревья обучаются независимо и параллельно, а затем их результаты просто усредняются) и метод случайных подпространств (разбиение объектов в вершине случайного дерева ищется среди случайного подмножества признаков, а обучение каждого дерева происходит на выборке, полученной с помощью операции бутстрепа- случайной генерации выборок).
- Метод основан на использовании большого **(комитета) ансамбля решающих деревьев**, каждое из которых само по себе даёт очень невысокое качество классификации, но за счёт их большого количества результат улучшается.

Случайный лес

Алгоритм обучения:

Обучающая выборка состоит из N образцов, размерность пространства признаков равна M ,

Построение ансамбля деревьев:

- Генерируется случайная подвыборка размером N из обучающей выборки. Некоторые образцы попадут в неё два или более раза, тогда как в среднем $N(1-\frac{1}{N})^N$ образцов не войдут в подвыборку.
- Формируется решающее дерево, классифицирующее образцы данной подвыборки, причём в ходе создания очередного узла дерева случайным образом выбирается m признаков из всех M признаков. Выбор наилучшего из этих m признаков может осуществляться различными способами.
- Дерево строится до полного исчерпания подвыборки и не подвергается процедуре прунинга (англ. Pruning) — отсечение ветвей),

Классификация объектов проводится путём голосования: каждое дерево комитета относит классифицируемый объект к одному из классов, а побеждает класс, за который проголосовало наибольшее число деревьев.

Оптимальное число деревьев подбирается таким образом, чтобы минимизировать ошибку классификатора на тестовой выборке. В случае её отсутствия, минимизируется оценка ошибки на не вошедших в набор образцах.

Характеристика деревьев решений

+ Просты в понимании и интерпретации.
+ Не требуют подготовки данных.
+ Используют модель «белого ящика».
+ Позволяют оценить модель при помощи статистических тестов.
+ Дают возможность извлекать из базы данных правила на естественном языке.
+ Позволяют создавать классификационные модели в тех областях, где аналитику достаточно сложно формализовать знания.
+ Алгоритм конструирования дерева решений не требует от пользователя выбора входных атрибутов.
+ Быстро обучаются.

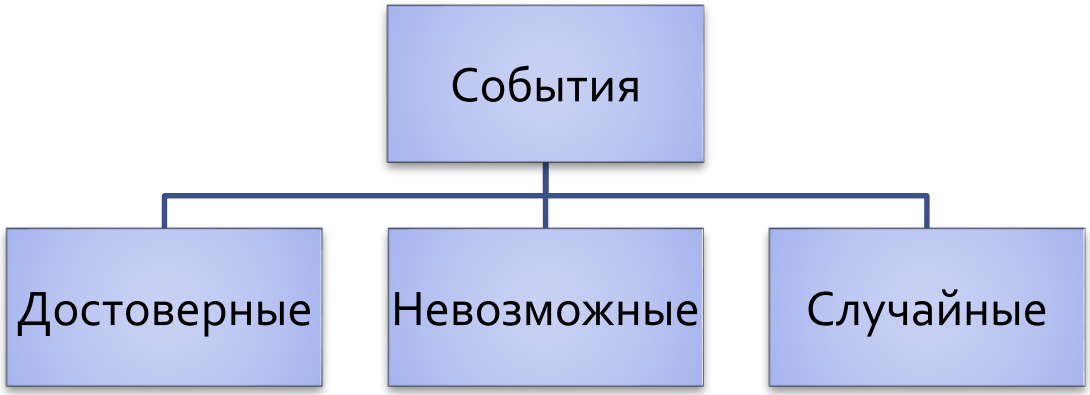
– Проблема получения оптимального дерева решений бывает *NP*-полной.
– Могут появиться слишком сложные конструкции, которые при этом недостаточно полно представляют данные.
– Существуют концепты, которые сложно понять из модели, так как модель описывает их сложным путём.
– Для данных, которые включают категориальные переменные с большим набором уровней, больший информационный вес присваивается тем атрибутам, которые имеют большее количество уровней.

Характеристика случайного леса

+ Высокая точность предсказания, которая сравнима с результатами градиентного бустинга.
+ Эффективная обработка данных с большим числом признаков и классов.
+ Нечувствительность к масштабированию (и вообще к любым монотонным преобразованиям) значений признаков.
+ Использует методы оценивания значимости отдельных признаков в модели.
+ Внутренняя оценка способности модели к обобщению (тест по неотобранным образцам).
+ Высокая параллелизуемость и масштабируемость.

– Большой размер получающихся моделей.
– Очень трудоемкий процесс прогнозирования по сравнению с другими алгоритмами.
– Алгоритм склонен к переобучению на зашумленных данных.
– В отличие от более простых алгоритмов, результаты случайного леса сложнее интерпретировать.

Основы теории вероятностей: Виды событий

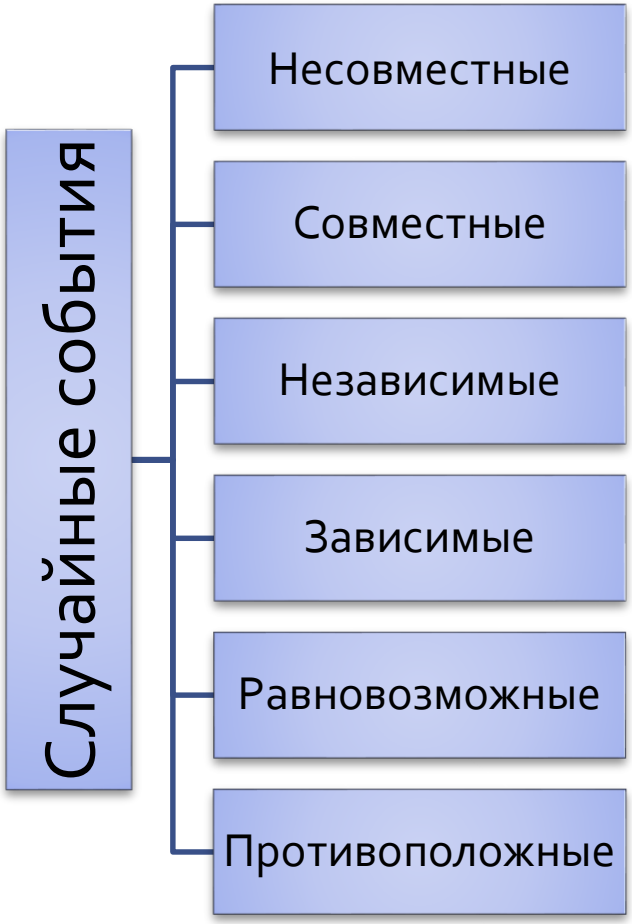


Достоверные события **всегда** происходят при осуществлении данной совокупности условий

Невозможные события **никогда не** происходят при осуществлении данной совокупности условий

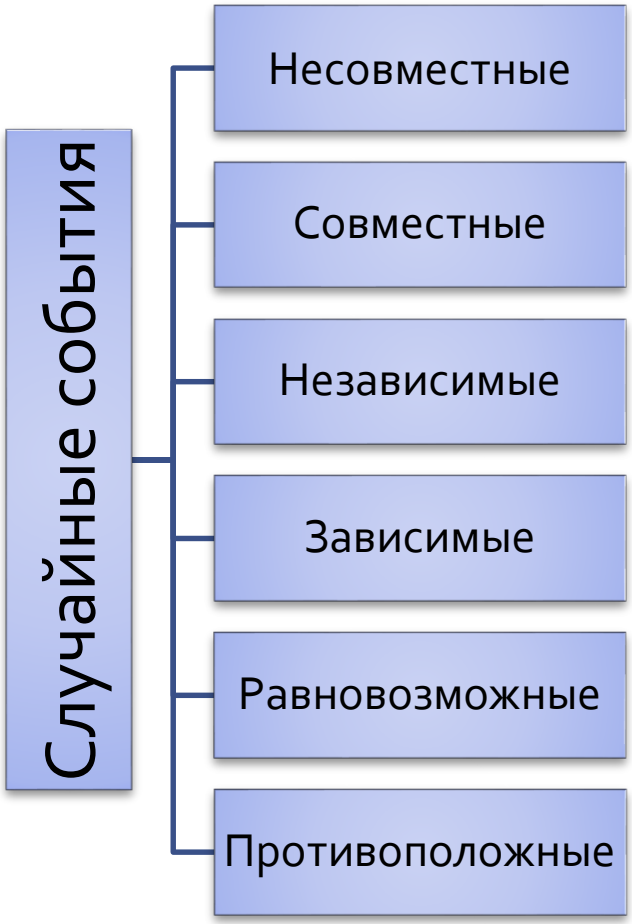
Случайные события **могут произойти** или **не произойти** при осуществлении данной совокупности условий

Основы теории вероятностей: Случайные события



- **Несовместными** называются события, которые не могут одновременно произойти в одном испытании
- Совокупность случайных событий A_1, A_2, A_3,... A_n называется **полной группой** для данного испытания, если в результате испытания обязательно происходит только одно из событий этой совокупности
- Два события (A и $\bar{A}$) называются **противоположными**, если появление одного из них равносильно не появлению другого

Основы теории вероятностей: Случайные события



- **Совместными** называются события, которые могут одновременно произойти в одном испытании
- События называются **независимыми**, если появление одного из них не изменяет вероятности появления второго.
- События называются **зависимыми** если появление одного из них зависит от появления другого
- **Равновозможными** называются события, если ни у одного из них нет объективного преимущества перед другим

Основы теории вероятностей: Классическое определение вероятности

Вероятностью события A называют отношение числа благоприятствующих этому событию элементарных событий (m) к общему числу всех равновозможных несовместных элементарных событий (n), образующих полную группу:

$$P(A) = \frac{m}{n}$$

Чтобы рассчитать классическую вероятность необходимо до проведения испытаний теоретически подсчитать:

- общее число всех **равновозможных несовместных** элементарных событий (n)
- число **благоприятствующих** этому событию равновозможных несовместных элементарных событий (m)

- **Вероятность** достоверного **события** $P = 1$
- **Вероятность** невозможного **события** $P = 0$
- **Вероятность** случайного **события** $0 < P < 1$

Условная вероятность

Определение.

Пусть $P(A)>0$.

Условной вероятностью *P(B/A)* события *B* при условии, что событие *A* наступило, называется число

$$P(B/A)=\frac{P(AB)}{P(A)}$$

Обозначения:

$$P(B/A)=P_A(B)$$

Условная вероятность удовлетворяет всем аксиомам вероятности.

В частности,

$$0\le P(B/A)\le 1,\ \ P(A/A)=1$$

Независимые события

Определение.

- События A и B называются **независимыми**, если

$$P(AB) = P(A)P(B)$$

Определение. Пусть $P(A)>0$ и $P(B)>0$.

- **Событие *A* не зависит от *B***, если $P(A/B) = P(A)$

Следствие.

- Если событие A не зависит от B, то и событие B не зависит от A.

$$P(AB) = P(A/B)P(B) = P(A)P(B)$$

- Доказательство.

$$P(B/A) = \frac{P(AB)}{P(A)} = \frac{P(A)P(B)}{P(A)} = P(B)$$

На практике из физической независимости событий делают вывод о теоретико-вероятностной независимости.

Полная группа событий

- События **образуют** **полную группу**, если они
- 1) попарно несовместны
- 2) в результате эксперимента обязательно какое- либо одно из них наступит

$$P(H_i H_j) = 0, \ i \neq j$$

$H_1 + H_2 + ,\ldots, + H_n = \Omega$ H_i - **гипотезы**

Пример.

В стохастическом эксперименте рассмотрим события A и $\overline{A}$

Они образуют полную группу.

Формула полной вероятности

Теорема.

- Если события $H_1, H_2, \ldots, H_n$

образуют **полную группу**,

то для любого события A справедлива формула

$$P(A) = P(H_1)P(A/H_1) + \ldots + P(H_n)P(A/H_n)$$

$$P(A) = \sum_{i=1}^{n} P(H_i)P(A/H_i)$$

Формула Байеса

Теорема.

- Пусть события $H_1, H_2, ..., H_n$

образуют **полную группу.**

Пусть событие ***A*** наступило ***(P(A)>0)***.

Тогда вероятность того,

что при этом была реализована гипотеза (наступило событие) H_k

вычисляется по формуле

$$P(H_k/A) = \frac{P(H_k)P(A/H_k)}{P(A)} = \frac{P(H_k)P(A/H_k)}{\sum_{i=1}^{n} P(H_i)P(A/H_i)}$$

Формула Байеса позволяет переоценить вероятности гипотез после того, как проведено испытание, в результате которого произошло событие *A*.

Формула Байеса. Частный случай

- Рассмотрим события H и $\overline{H}$

они образуют **полную группу.**

Пусть событие A наступило $(P(A)>0)$.

Тогда вероятность того,

что при этом была реализована гипотеза H

вычисляется по формуле

$$P(H/A)=\frac{P(H)P(A/H)}{P(A)}$$

$$P(H/A)=\frac{P(H)P(A/H)}{P(H)P(A/H)+P(\overline{H})P(A/\overline{H})}$$

Вероятностная формулировка задачи машинного обучения

- Эмпирический риск:

$$R_{Emp}(a, X^l) = P\left(a(x) \neq y \mid X^l\right) = \frac{1}{l}\sum_{i=1}^{l}[a(x_i) \neq y_i]$$

- Общий риск:

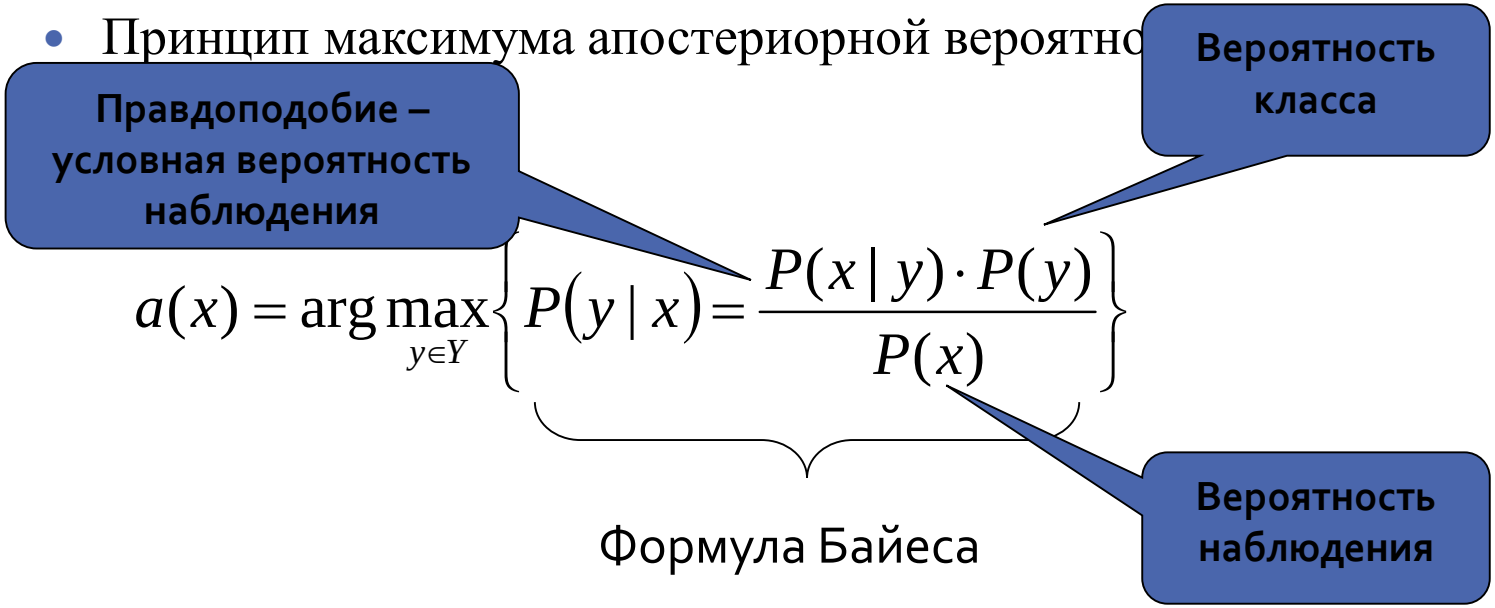
$$R(a, X) = P\left(a(x) \neq y \mid X\right) = \int_X P(x)\left[a(x) \neq y\right]dx$$

  - рассчитать невозможно
  - требуется минимизировать
- Модель алгоритма и метод обучения определяются так же

Наивный байесовский классификатор

Наи́вный ба́йесовский классифика́тор — простой вероятностный классификатор, основанный на применении Теоремы Байеса со строгими (наивными) предположениями о независимости.

- Предположения:
  - Известна функция правдоподобия: $P(x \mid y)$
  - Известны априорные вероятности: $P(y), P(x)$
- Принцип максимума апостериорной вероятно



Наивный байесовский классификатор

Алгоритм:

1. Для каждой гипотезы вычислить апостериорную вероятность.
2. *Выбрать гипотезу с максимальной апостериорной вероятностью*

Эмпирический риск: $R_{emp}(a, X) = P(a(x) \neq y \mid X)$

Особенности наивного байесовского классификатора:

- Нужно знать функцию правдоподобия и априорные вероятности
- *Отсутствуют априорные причины верить, что одна из гипотез более вероятна чем другая (наивность)*
- *Отвечает на вопрос – Какова наиболее вероятная гипотеза при имеющихся данных?*
- *Надо ответить на вопрос – Какова наиболее вероятная классификации нового примера при имеющихся данных?*